

Memory Hierarchy & Cache Fundamentals

Most developers know how to write `@Cacheable`, but when an interviewer asks, "Why can't PostgreSQL handle 10,000 read requests per second even with indexes?", they get stuck.

To understand this, we need to understand why database is slow, why RAM is incredibly fast and how the Cache-Aside pattern operates behind the scenes.

1. The Real Bottleneck: Disk I/O vs Network vs RAM

Computers operate on a strict memory hierarchy. The further data sits from the CPU, the slower it is to access.



- **CPU L1 / L2 Cache:** ~0.5 to 7 nanoseconds
- **RAM Access:** ~50 to 100 nanoseconds
- **NVMe / SSD Read:** ~50 to 150 microseconds (1,000 times slower than RAM)
- **Hard Disk Drive Seek:** ~1 to 10 milliseconds (100,000 times slower than RAM)
- **Network Call to a Database:** ~2 to 30 milliseconds (depending on query complexity and network hop)

2. Why Relational Databases Slow Down on Reads

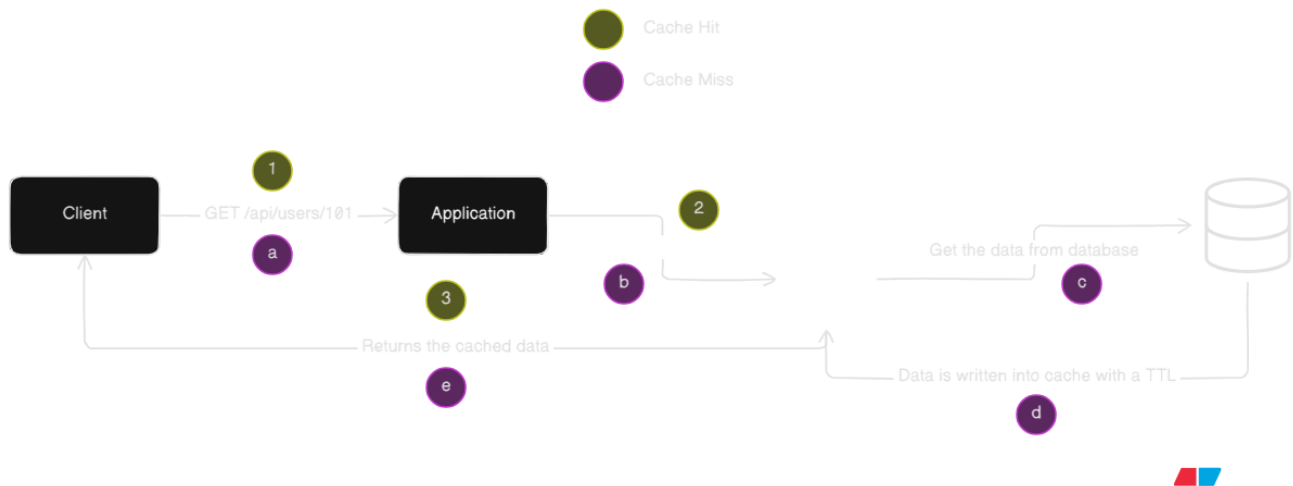
Even if your SQL query is simple, the database engine must perform several heavy operations:

1. **Connection Overhead:** Parsing the connection from the pool, authenticating, and thread allocation.
2. **Query Parsing & Execution Planning:** Parsing SQL syntax, verifying permissions, and checking the query planner.
3. **Disk & Page Buffer Traversal:** Traversal of B-Tree indexes on disk or reading data pages into the database's internal buffer pool.
4. **Concurrency Locking:** Managing MVCC (Multi-Version Concurrency Control) and row/table locks to prevent dirty reads.
5. **Serialization:** Converting raw database rows into JDBC ResultSets, which Spring converts into Java Objects.

When 1,000 users hit that query simultaneously, the database runs out of connection pool threads and disk throughput, driving CPU usage to 100%.

3. The Core Solution: Cache-Aside Pattern (Lazy Loading)

Cache-Aside is the default pattern used in 95% of Spring Boot applications (this is what `@Cacheable` does behind the scenes).



Step-by-step lifecycle:

- **Step 1:** Application receives a request (e.g., Get User by ID 101).
- **Step 2:** Check Redis first using key users:101.
- **Step 3 (Branch A - Cache Hit):**
 - Data exists in Redis.
 - Return the cached JSON/Object directly to the user.
 - Time taken: ~2 to 5 milliseconds.
 - Database is never touched.
- **Step 4 (Branch B - Cache Miss):**
 - Data does not exist in Redis (null).
 - Application queries the database (PostgreSQL/MySQL).
 - Database returns the entity/record.
 - Application converts entity to DTO.
 - Application writes the DTO to Redis with a TTL (Time-To-Live).
 - Application returns the response to the user.
 - Next time anyone requests ID 101, it takes Branch A.

4. Other Cache Patterns

Sr. No.	Cache Pattern	Description	Advantage	Disadvantage
1.	Write-Through Cache	Application writes to the cache and the database at the exact same time.	Cache is never stale.	Writes take longer because you are writing to two systems.

Sr. No.	Cache Pattern	Description	Advantage	Disadvantage
2.	Write-Behind (Write-Back) Cache	Application writes only to Redis immediately and acknowledges the user. A background worker batches those writes and updates the database later.	Lightning-fast writes.	Risk of data loss if Redis crashes before the background write flushes to the database.